

Lab 3 - Component Modelling & Architectural Pattern Selection

Name: AMOGH SHARMA | SRN: PES1UG24CS053 | Course: Software Engineering (Sem 5)
Problem Statement #53 - Freelance Content Creator Escrow Platform

Architecture Selection: We chose the MICROSERVICES ARCHITECTURE for the Freelance Content Creator Escrow Platform.

Style	Fit for this scenario	Verdict
Layered	Clean separation, but payments, media watermarking and contract logic would ship as one deployable unit - a slow media job degrades the whole platform and the PCI audit scope covers every layer.	Rejected
Client-Server	Simple and consistent, but a single server is one point of failure for escrowed money and cannot absorb bursts of large media uploads.	Rejected
Microservices	Money path, media path and identity are separately deployable, separately scalable and separately secured - exactly the three axes on which this platform differs.	Selected

1. Architectural Choice

The system is decomposed into five independently deployable services - Contract & Milestone Manager, Escrow & Payment Service, Asset & Watermarking Service, Identity & Access Service and Notification Service - fronted by an API Gateway. Each service owns its own data store and is reached only through a published interface (IContractStore, IPaymentProcessing, IObjectStore, IAuthValidation, INotifyChannel), so no service reads another service's tables directly.

2. Reason 1 - the workloads are asymmetric

Watermarking every draft deliverable (FR-003: creator name, contract ID and timestamp burned into each preview) is CPU- and I/O-heavy and arrives in bursts when creators submit, while contract and milestone operations are light CRUD calls. Microservices let the Asset & Watermarking Service scale horizontally on its own during an upload burst, instead of over-provisioning the entire platform to survive a media spike.

3. Reason 2 - the money path must fail independently

The escrow guarantee (FR-001, FR-002: funds release only after sponsor sign-off, contracts go ACTIVE only when fully funded) is the behaviour that makes the platform trustworthy. Isolating the Escrow & Payment Service means an outage in preview generation or notification delivery cannot corrupt or block a milestone state transition; the failure is contained to one service instead of taking the whole deployable down with it.

4. Security Advantage - a compartmentalised, narrow trust boundary

Card data and payment-gateway credentials live only inside the Escrow & Payment Service and are reachable only through the IPaymentProcessing interface. The Asset & Watermarking Service - the component that handles untrusted, user-uploaded media and is therefore the most likely to be exploited - holds no payment secrets and has no network path to the payment gateway, so compromising it cannot reach funds. Authentication and role checks (Creator / Sponsor / Admin) are enforced once at the API Gateway through IAuthValidation, and original un-watermarked assets stay behind the Asset Service's 403 gate until the milestone is PAID. This also shrinks the PCI-DSS audit scope to a single service rather than the whole application.

5. Performance Benefit - heavy work stays off the critical path

Watermark and preview generation runs on separately scaled Asset Service instances and is triggered asynchronously via events, so a sponsor loading the review page and a creator submitting a draft are not queued behind each other. The p95 latency of contract and escrow API calls therefore stays flat even while large media files are being processed - something a single layered deployment, where all requests share one thread pool and one process, cannot promise.

6. Trade-off Acknowledged

Microservices cost operational complexity and give up strong cross-service consistency. This is mitigated by keeping the milestone state machine authoritative in exactly one service (Contract & Milestone Manager) and by making IPaymentProcessing idempotent, so a retried release can never pay a creator twice.